Simulation and Modelling

NATIONAL UNIVERSITY
of Computer & Emerging Sciences

Spring 2025
CS4056

Muhammad Shahid Ashraf

Notes

Monte Carlo Simulations

Notes

The first thoughts and attempts I made to practice [the Monte Carlo Method] were suggested by a question which occurred to me in 1946 as I was convalescing from an illness and playing solitaires. The question was what are the chances that a Canfield solitaire laid out with 52 cards will come out successfully? After spending a lot of time trying to estimate them by pure combinatorial calculations, I wondered whether a more practical method than "abstract thinking" might not be to lay it out say one hundred times and simply observe and count the number of successful plays. This was already possible to envisage with the beginning of the new era of fast computers, and I immediately thought of problems of neutron diffusion and other questions of mathematical physics, and more generally how to change processes described by certain differential equations into an equivalent form interpretable as a succession of random operations. Later ... [in 1946, I] described the idea to John von Neumann, and we began to plan actual calculations.

Notes

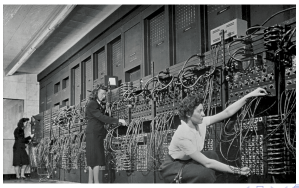
Monte Carlo Simulation

- Monte Carlo Simulation, also known as the Monte Carlo Method or a multiple probability simulation.
- MCS is a mathematical technique, which is used to estimate the possible outcomes of an uncertain event.
- The Monte Carlo Method was invented by John von Neumann and Stanislaw Ulam during World War II to improve decision making under uncertain conditions.
- It was named after a well-known casino town, called Monaco, since the element of chance is core to the modeling approach, similar to a game of roulette.

Notes

Monte Carlo Simulation

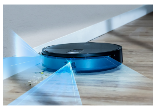
- Ulam, recovering from an illness, was playing a lot of solitaire
- Tried to figure out probability of winning, and failed
- Thought about playing lots of hands and counting number of wins, but decided it would take years
- Asked Von Neumann if he could build a program to simulate many hands on ENIAC



Notes

Monte Carlo Simulation

- A method of estimating the value of an unknown quantity using the principles of inferential statistics
- Inferential statistics
  - Population: a set of examples
  - Sample: a proper subset of a population
  - Key fact: a random sample tends to exhibit the same properties as the population from which it is drawn
- Random Walk of AI cleaner



Notes

Example

- Given a single coin, estimate fraction of heads you would get if you flipped the coin an infinite number of times
- Consider one flip,How confident would you be about answering 1.0?
- Flipping a Coin Twice, Do you think that the next flip will come up heads?
- Flipping a Coin 100 Times, Now do you 0think that the next flip will come up heads?
- Do you think that the probability of the next flip coming up heads is 52/100?
- Given the data, its your best estimate But confidence should be low.

Notes

Why the Difference in Confidence?

- Confidence in our estimate depends upon two things
- Size of sample (e.g., 100 versus 2)
- Variance of sample (e.g., all heads versus 52 heads)
- As the variance grows, we need larger samples to have the same degree of confidence.

Notes

Empirical Rule

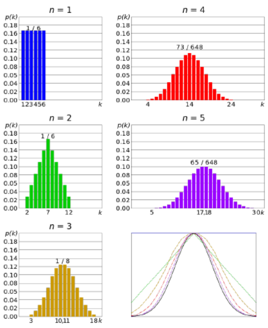
Under some assumptions discussed

- 68% of data within one standard deviation of mean
- 95% of data within 1.96 standard deviations of mean
- 99.7% of data within 3 standard deviations of mean
- But not all distribution are Normal
  - Empirical works for normal distribution
  - But are the outcomes of spins of a roulette wheel normaly distributed
  - No they are uniformly distributed(Each outcome is equally probable.
  - So, why does the empirical rule work here

Notes

Why Did the Empirical Rule Work?

- Because we are reasoning not about a single spin, but about the mean of a set of spins
- And the central limit theorem applies



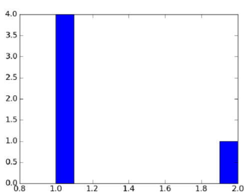
Notes

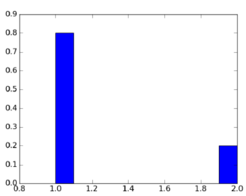
The Central Limit Theorem (CLT)

- Given a sufficiently large sample:
  1) The means of the samples in a set of samples (the sample means) will be approximately normally distributed,
  2) This normal distribution will have a mean close to the mean of the population, and
  3) The variance of the sample means will be close to the variance of the population divided by the sample size.

Notes

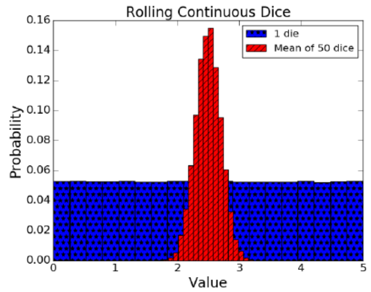
Weighting the Bins





Notes

Mean of rolling 1 die = 2.49759575528, Std = 1.4439045633
Mean of rolling 50 dice = 2.49985051798, Std = 0.204887274645



Notes

Try It for Roulette

NATIONAL UNIVERSITY
of Computer & Emerging Sciences

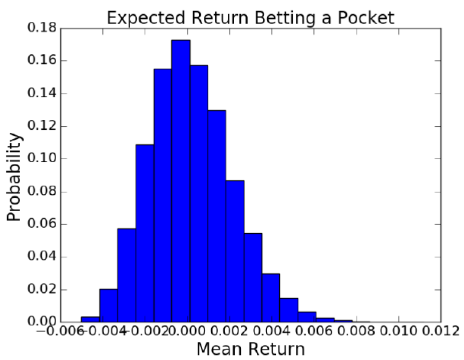
```
numTrials = 50000
numSpins = 200
game = FairRoulette()

means = []
for i in range(numTrials):
    means.append(findPocketReturn(game, 1,
  numSpins)[0]/numSpins)

pylab.hist(means, bins = 19,
           weights = pylab.array(len(means)*[1])/len(means))
pylab.xlabel('Mean Return')
pylab.ylabel('Probability')
pylab.title('Expected Return Betting a Pocket')
```


Notes

Means Close to Normally Distributed!



Notes

Try It for Roulette

NATIONAL UNIVERSITY
of Computer & Emerging Sciences

- It doesn't matter what the shape of the distribution of values happens to be
- If we are trying to estimate the mean of a population using sufficiently large samples
- The CLT allows us to use the empirical rule when computing confidence intervals

Notes

Gambler's Fallacy

- "On August 18, 1913, at the casino in Monte Carlo, black came up a record twenty six times in succession (in roulette)(There) was a near panicky rush to bet on red, beginning about the time black had come up a phenomenal fifteen times" Huff and Geis, How to Take a Chance
- Probability of 26 consecutive reds

$$\frac{1}{67,108,865}$$

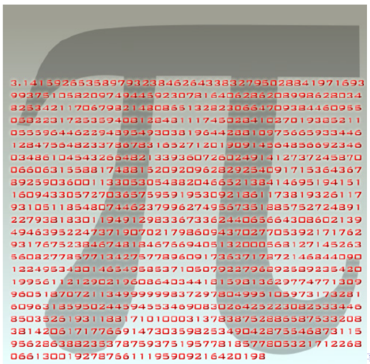
- Probability of 26 consecutive reds when previous 25 rolls were red

$$\frac{1}{2}$$

- Following an extreme random event, the next random event is likely to be less extreme
- If you spin a fair roulette wheel 10 times and get 100%reds, that is an extreme event

Notes

Estimating PI



Notes

Estimating PI

$$\frac{circumference}{diameter} = \Pi$$

$$area = \Pi * radius^2$$

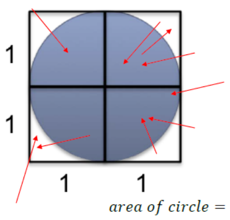
Notes

Bufoon-Laplace

- Buffon's needle problem is a question first posed in the 18th century by Georges-Louis Leclerc, Comte de Buffon
- Suppose we have a floor made of parallel strips of wood, each the same width, and we drop a needle onto the floor.
- What is the probability that the needle will lie across a line between two strips?
- Buffon's needle was the earliest problem in geometric probability to be solved;
- it can be solved using integral geometry. The solution for the sought probability p, in the case where the needle length l is not greater than the width t of the strips, is $p = \frac{2}{\pi}\frac{l}{t}$
- This can be used to design a Monte Carlo method for approximating the number π, although that was not the original motivation for de Buffon's question

Notes

Estimating PI Bufoon-Laplace



$A_s = 2*2 = 4$
$A_c = \pi r^2 = \pi$

$$\frac{needles\ in\ circle}{needles\ in\ square} = \frac{area\ of\ circle}{area\ of\ square}$$

$$area\ of\ circle = \frac{area\ of\ square * needles\ in\ circle}{needles\ in\ square}$$

$$area\ of\ circle = \frac{4 * needles\ in\ circle}{needles\ in\ square}$$

Notes

Arrows Are More Fun than Needles

Notes

Simulating Buffon-Laplace Method

```
def throwNeedles(numNeedles):
    inCircle = 0
    for Needles in range(1, numNeedles + 1, 1):
        x = random.random()
        y = random.random()
        if (x*x + y*y)**0.5 <= 1.0:
            inCircle += 1
    return 4*(inCircle/float(numNeedles))
```


Notes

Simulating Buffon-Laplace Method

```
def getEst(numNeedles, numTrials):
    estimates = []
    for t in range(numTrials):
        piGuess = throwNeedles(numNeedles)
        estimates.append(piGuess)
    sDev = numpy.std(estimates)
    curEst = sum(estimates)/len(estimates)
    print('Est. = ' + str(curEst) +\
          ', Std. dev. = ' + str(round(sDev, 6))\
          + ', Needles = ' + str(numNeedles))
    return (curEst, sDev)
```


Notes

Estimating PI

```
def estPi(precision, numTrials):
    numNeedles = 1000
    sDev = precision
    while sDev >= precision/1.96:
        curEst, sDev = getEst(numNeedles, numTrials)
        numNeedles *= 2
    return curEst
```


Notes

Estimating PI

```
Est. = 3.1484400000000012, Std. dev. = 0.047886, Needles = 1000
Est. = 3.1391799999999987, Std. dev. = 0.035495, Needles = 2000
Est. = 3.1410799999999997, Std. dev. = 0.02713, Needles = 4000
Est. = 3.141435, Std. dev. = 0.016805, Needles = 8000
Est. = 3.141355, Std. dev. = 0.0137, Needles = 16000
Est. = 3.1413137500000006, Std. dev. = 0.008476, Needles = 32000
Est. = 3.141171874999999, Std. dev. = 0.007028, Needles = 64000
Est. = 3.1415896874999993, Std. dev. = 0.004035, Needles = 128000
Est. = 3.1417414062499995, Std. dev. = 0.003536, Needles = 256000
Est. = 3.14155671875, Std. dev. = 0.002101, Needles = 512000
```


Notes

Being Right is Not Good Enough

- Not sufficient to produce a good answer
- Need to have reason to believe that it is close to right
- In this case, small standard deviation implies that we are close to the true value of π

Right?

Notes

Is it Correct to State

- 95% of the time we run this simulation, we will estimate that the value of pi is between 3.13743875875 and 3.14567467875?
- With a probability of 0.95 the actual value of π is between 3.13743875875 and 3.14567467875?
- Both are factually correct
- But only one of these statement can be inferred from our simulation
- $statiscally\ valid \neq true$

Notes

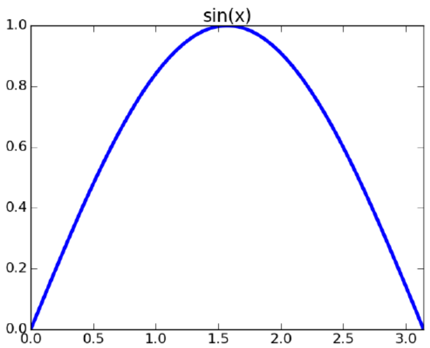
Generally Useful Technique

- To estimate the area of some region, R
  - Pick an enclosing region, E, such that the area of E is easy to calculate and R lies completely within E
  - Pick a set of random points that lie within E
  - Let F be the fraction of the points that fall within R
  - Multiply the area of E by F
- Way to estimate integrals

$$\int_0^{\pi} \sin(x)$$

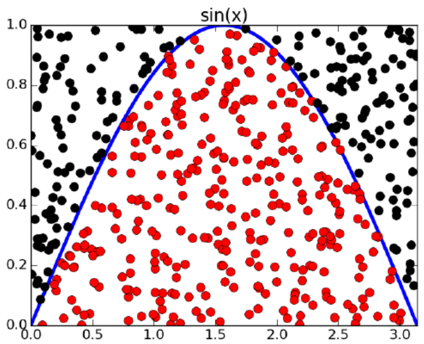
Notes

Sin X



Notes

Random Points



Notes

Notes